

Name: S.SANJANA

Roll no: 2403A510D4

Course: AI assisted coding

Question:

Task1:

Prompt: give me the code to identify the tweet is positive,negative or nutral with a zero-shot prompt to classify sentiment without any examples

Code:

```
from transformers import pipeline

# Load zero-shot classification pipeline
classifier = pipeline("zero-shot-classification", model="facebook/bart-large-mnli")

def classify_sentiment(tweet):
    candidate_labels = ["positive", "negative", "neutral"]
    result = classifier(tweet, candidate_labels)
    return result['labels'][0]

# Example usage
tweet = "I love using this new app!"
sentiment = classify_sentiment(tweet)
print(f"Sentiment: {sentiment}")
```

Output:

```
tweet = "I love using this new app!"
Sentiment: positive
```

Observstion:

1. Uses Zero-Shot Classification Pipeline

- Leverages Hugging Face's pipeline for zero-shot classification.
- No fine-tuning needed; uses pretrained facebook/bart-large-mnli model.

2. Sentiment Classification with Custom Labels

- Classifies input tweet into one of three categories: "positive", "negative", or "neutral".

3. Model is NLI-Based

- The model is originally trained for Natural Language Inference but adapted here for sentiment by comparing the tweet against candidate labels

4. Returns the Top Predicted Label

- Only the label with the highest confidence (`result['labels'][0]`) is returned and printed.

5. Simple and Reusable Function

- The `classify_sentiment` function takes any tweet and returns its sentiment label, making it easy to use for multiple inputs.

6. Requires External Dependencies

- Needs `transformers` library installed.
- Downloads the model the first time (internet connection required).

Task2:

Prompt 1:

Solve the following math problem step by step:

"{math_problem}"

Code:

```
def generate_math_prompt(math_problem):  
    prompt = f'Solve the following math problem step by step:\n"{math_problem}"'  
    return prompt  
  
# Example usage:  
problem = "3x + 5 = 20"  
print(generate_math_prompt(problem))
```

output:

```
Solve the following math problem step by step:  
"3x + 5 = 20"
```

Observations:

1. Clear Function Purpose

- The function `generate_math_prompt` is designed to generate a **prompt** that asks for a **step-by-step solution** to a math problem.
 - It is likely intended for use with **AI assistants**, such as ChatGPT or other language models.
-

2. Good Use of F-String

- The f-string formatting (`f'String {variable}'`) is a **clean and efficient** way to insert the `math_problem` into the prompt.

3. Reusable and Simple

- The function is simple and can easily be reused for any math problem.
 - You can change `math_problem` to generate prompts for different equations or expressions.
-

4. No Input Validation

- There's **no check** to ensure that the input is a valid or non-empty string.
 - Example: If `math_problem = ""`, the output will be:
 - Solve the following math problem step by step:
 - ""
 - This is technically valid but not very useful.

5. Readable and Beginner-Friendly

- The code is clean, short, and easy to understand — great for beginners learning about functions and string formatting.

Prompt 2:

You are an AI math tutor helping a 7th-grade student who is learning to solve linear equations. The student finds problems of moderate difficulty challenging and needs clear, simple explanations. Please solve the following problem step-by-step and explain each step:

"12x - 4 = 20 give me the code

Code:

```
def generate_detailed_math_prompt(math_problem, grade_level="7th grade", topic="solving linear equations",
difficulty="moderate"):
    prompt = (
        f'You are an AI math tutor helping a {grade_level} student who is learning to {topic}. '
        f'The student finds problems of {difficulty} difficulty challenging and needs clear, simple explanations. '
        f'Please solve the following problem step-by-step and explain each step:\n\n"{math_problem}"'
    )
    return prompt

# Example usage
problem = "12x - 4 = 20"
print(generate_detailed_math_prompt(problem))
```

output:

```
You are an AI math tutor helping a 7th grade student who is learning to solve linear equations. The student finds
problems of moderate difficulty challenging and needs clear, simple explanations. Please solve the following problem
step-by-step and explain each step:

"12x - 4 = 20"
```

Observation:

1. Defines a Simple Prompt Generator Function

- The function `generate_math_prompt()` takes one input: `math_problem`.
- It returns a formatted prompt string asking to solve the problem step-by-step.

2. Dynamic Input Handling

- You can reuse the function with **any math problem** just by changing the input string.

- Example: You can plug in " $x^2 + 2x - 3 = 0$ " or " $7 + 3 \times 2$ " as well.

3. Useful for Prompt Engineering

- This kind of prompt is especially useful for LLMs (e.g., GPT models).
- It guides the model to give **step-by-step** reasoning rather than just an answer.

4. Good Readability

- The function is simple, readable, and cleanly formatted.
- Uses an f-string for efficient string formatting.

5. Basic, No Validation or Error Handling

- There's **no validation** of whether the input is a valid math problem.
- If someone passes an empty string or a non-math string, it still returns a prompt without errors.

QUESTION 2:

Task1:

Prompt: A one-shot prompt (give 1 example of classification).

A few-shot prompt (give 3–4 examples) give me code?

Code:

```

vscode > AI > # -----
1 def classify_tweet_sentiment(tweet: str) -> str:
2     positive_keywords = ['good', 'great', 'happy', 'love', 'excellent', 'awesome',
3     'fantastic', 'amazing']
4     negative_keywords = ['bad', 'sad', 'hate', 'terrible', 'awful', 'worst',
5     'horrible', 'disappoint']
6
7     tweet_lower = tweet.lower()
8     if any(word in tweet_lower for word in positive_keywords):
9         return "Positive"
10    elif any(word in tweet_lower for word in negative_keywords):
11        return "Negative"
12    else:
13        return "Neutral"
14
15    # -----
16    # Task 1: One-shot Prompt Example
17    # -----
18    print("One-shot Example:")
19    example_1 = "I had an awesome day!"
20    print(f"Tweet: '{example_1}'")
21    print(f"Sentiment:", classify_tweet_sentiment(example_1))

```

```

print("\nFew-shot Examples:")
examples = [
    "I love this new phone, it's amazing!",    # Positive
    "This weather is just terrible and awful.", # Negative
    "I'm going to the store later.",            # Neutral
    "The movie was good but the ending was bad." # Positive (due to order of checks)
]

for i, tweet in enumerate(examples, 1):
    print(f"\nExample {i}:")
    print(f"Tweet: '{tweet}'")
    print(f"Sentiment:", classify_tweet_sentiment(tweet))

```

Output:

```
One-shot Example:  
Tweet: 'I had an awesome day!'  
Sentiment: Positive  
Example 1:  
Tweet: 'I love this new phone, it's amazing!'  
Sentiment: Positive
```

Observations:

```
Converts tweets to lowercase for case-insensitive matching  
Checks for positive or negative keywords as substrings in the tweet |  
Returns "Positive", "Negative", or "Neutral" based on keyword presence  
Prioritizes positive sentiment if both are found, which can misclassify
```

TASK2:

PROMPT: Compare outputs on the same set of tweets and explain the difference give me code

CODE:

```
from transformers import pipeline  
  
# Load the zero-shot classification model  
classifier = pipeline("zero-shot-classification", model="facebook/bart-large-mnli")  
  
# Function to classify sentiment and return full result  
def classify_sentiment(tweet):  
    candidate_labels = ["positive", "negative", "neutral"]  
    result = classifier(tweet, candidate_labels)  
    return result  
  
# List of sample tweets  
tweets = [  
    "I love using this new app!",  
    "This service is terrible and frustrating.",  
    "The meeting happened today as scheduled.",  
    "I'm not sure how I feel about this product.",  
    "Absolutely amazing performance by the team!"  
]
```

```
# Analyze each tweet
for i, tweet in enumerate(tweets, start=1):
    result = classify_sentiment(tweet)
    print(f"\nTweet {i}: {tweet}")
    for label, score in zip(result['labels'], result['scores']):
        print(f"    {label.capitalize()}: {score:.4f}")
    print(f">>> Predicted Sentiment: {result['labels'][0].upper()}")
```

OUTPUT:

```
Tweet 1: I love using this new app!
    Positive: 0.9532
    Neutral: 0.0345
    Negative: 0.0123
>>> Predicted Sentiment: POSITIVE

Tweet 2: This service is terrible and frustrating.
    Negative: 0.9448
    Neutral: 0.0387
    Positive: 0.0165
>>> Predicted Sentiment: NEGATIVE

Tweet 3: The meeting happened today as scheduled.
    Neutral: 0.8062
    Positive: 0.1129
    Negative: 0.0809
>>> Predicted Sentiment: NEUTRAL

Tweet 4: I'm not sure how I feel about this product.
    Neutral: 0.5813
    Negative: 0.3092
    Positive: 0.1095
>>> Predicted Sentiment: NEUTRAL

Tweet 5: Absolutely amazing performance by the team!
    Positive: 0.9671
    Neutral: 0.0218
    Negative: 0.0111
>>> Predicted Sentiment: POSITIVE
```

Observations:

1. Zero-Shot Classification is Semantic, Not Trained for Sentiment Specifically

The model is not fine-tuned specifically for sentiment analysis but can classify based on how semantically close the tweet is to each label ("positive", "negative", "neutral").

This can work well in many cases but may lead to ambiguous or less accurate results for subtle or sarcastic tweets.

2. Label Ordering Affects Scores Slightly

The order of candidate_labels can sometimes slightly influence score distribution due to model internals. It's often negligible but worth knowing.

3. Neutral Statements May Be Misclassified

Tweets like: "The meeting happened today as scheduled."

Are objectively neutral, but the model might classify them as positive or negative depending on context and interpretation.

4. Confidence Scores Are Useful

The output:

python

Copy code

```
for label, score in zip(result['labels'], result['scores']):  
    print(f" {label.capitalize()}: {score:.4f}")
```

Helps understand how confident the model is in its prediction.

E.g., a score like:

makefile

Copy code

Positive: 0.50

Neutral: 0.48

Negative: 0.02

Shows ambiguity between positive and neutral.

5. Model Handles Strongly Worded Tweets Well

Example:

python

Copy code

"Absolutely amazing performance by the team!"

Is likely to return:

makefile

Copy code

Positive: ~0.9